DÉMO 004-D · CHANGEMENTS ENTRE LES ÉTAPES

Network status

Chaque étape comparée à la précédente : ce qui est ajouté et ce qui est retiré.

`+ ligne ajoutée` `– ligne retirée` `@@ position dans le fichier`

1. **Étape 1** · etape-1-listeners (3 fichiers, en entier)
2. **Étape 1 → 2** · etape-2-use-network-status (2 fichiers)

Étape 1 etape-1-listeners : point de départ

Les listeners online / offline dans un useEffect de Layout, avec leur nettoyage dans le return

Le code de départ, en entier : les fichiers écrits pour la démo et utilisés (atteints par les imports depuis src/main.*) ; les fichiers du gabarit Vite non modifiés sont seulement listés. Ordre de lecture : main → App → composants et pages → le reste.

Styles fournis (copiés tels quels, pas tapés en live) : src/App.css (100 lignes)

Fichiers générés par Vite, non modifiés (non affichés) : index.html, vite.config.js, .gitignore, eslint.config.js, public/vite.svg, package.json

DÉPART **src/main.jsx** 6 lignes

```jsx
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <App />
);
```

DÉPART **src/App.jsx** 8 lignes

```jsx
import "./App.css";
import Layout from "./components/Layout";

function App() {
  return <Layout />;
}

export default App;
```

DÉPART **src/components/Layout.jsx** 39 lignes

```jsx
import { useEffect, useState } from "react";

function Layout() {
  // navigator.onLine : true si le navigateur est connecté
  const [isOnline, setIsOnline] = useState(() => navigator.onLine);

  useEffect(() => {
    const onOnline = () => setIsOnline(true);
    const onOffline = () => setIsOnline(false);

    // Listeners : le navigateur nous prévient quand la connexion revient ou se coupe
    window.addEventListener("online", onOnline);
    window.addEventListener("offline", onOffline);

    // Nettoyage : on retire les écouteurs quand le composant disparaît
    return () => {
```

```
      window.removeEventListener("online", onOnline);
      window.removeEventListener("offline", onOffline);
    };
  }, []);

  return (
    <>
      {!isOnline && (
        <div style={{ padding: 10, background: "#ffdddd", color: "#900" }}>
          Vous êtes hors ligne
        </div>
      )}

      {isOnline && (
        <div style={{ padding: 10, background: "#ddffdd", color: "#090" }}>
          Vous êtes connecté
        </div>
      )}
    </>
  );
}

export default Layout;
```

Étape 1 → 2 etape-1-listeners → etape-2-use-network-status

La même logique sort dans le custom hook useNetworkStatus ; Layout ne garde que l'affichage

MODIFIÉ `src/components/Layout.jsx` +2 −18

```
@@ -1,23 +1,7 @@
- import { useEffect, useState } from "react";
+ import useNetworkStatus from "../hooks/useNetworkStatus";

  function Layout() {
-   // navigator.onLine : true si le navigateur est connecté
-   const [isOnline, setIsOnline] = useState(() => navigator.onLine);
-
-   useEffect(() => {
-     const onOnline = () => setIsOnline(true);
-     const onOffline = () => setIsOnline(false);
-
-     // Listeners : le navigateur nous prévient quand la connexion revient ou se coupe
-     window.addEventListener("online", onOnline);
-     window.addEventListener("offline", onOffline);
-
-     // Nettoyage : on retire les écouteurs quand le composant disparaît
-     return () => {
-       window.removeEventListener("online", onOnline);
-       window.removeEventListener("offline", onOffline);
-     };
-   }, []);
+   const { isOnline } = useNetworkStatus();

    return (
      <>
```

AJOUTÉ `src/hooks/useNetworkStatus.js` +22 −0

```
@@ -1,0 +1,22 @@
+ import { useEffect, useState } from "react";
+
+ function useNetworkStatus() {
+   const [isOnline, setIsOnline] = useState(() => navigator.onLine);
+
+   useEffect(() => {
+     const onOnline = () => setIsOnline(true);
+     const onOffline = () => setIsOnline(false);
+
+     window.addEventListener("online", onOnline);
+     window.addEventListener("offline", onOffline);
+
+     // Nettoyage : on retire les écouteurs quand le composant disparaît
+     return () => {
+       window.removeEventListener("online", onOnline);
+       window.removeEventListener("offline", onOffline);
+     };
+   }, []);
+
+   return { isOnline };
+ }
+ export default useNetworkStatus;
```